

Подзапросы. СТЕ. Расширенное понятие о представлениях



Балльно-рейтинговая система дисциплины "Системы управления базами данных"

Поток ПМ23-1, ПМ23-2, ПМ23-3, ПМ23-4, ПМ23-5, ПМ23-6,
ИД23-1, ИД23-2, ИД23-3, ПИ23-1, ПИ23-2

№ п/п	Вид отчетности	Баллы
1.	Работа в семестре – всего, в т.ч.:	40
1.1	Работа в первом модуле – всего, в т.ч.	20
1.1.1	Выполнение и защита компьютерных заданий	1,5-3 12
1.1.2	Выполнение самостоятельной работы	3 3
1.1.3	Посещения	0,294 5
1.2	Работа во втором модуле– всего, в т.ч.	20
1.2.1	Выполнение и защита компьютерных заданий	2 12
1.2.2	Выполнение контрольной работы	3 3
1.2.3	Посещения	0,294 5
2.	Экзамен	60
	Итого:	100



Календарный план дисциплины «Системы управления базами данных»

2 семестр, поток ПМ23-1, ПМ23-2, ПМ23-3, ПМ23-4, ПМ23-5, ПМ23-6,

ИД23-1, ИД23-2, ИД23-3, ПИ23-1, ПИ23-2



№	Название темы	Количество часов	№ Недели	Название занятия	Вид	Количество баллов	Посещения
		Модуль 1					
4	Формирование наборов данных: введение в оператор SELECT (продолжение)	2	1	Сложные запросы. Работа с представлениями. Продвинутые методы работы с SQL. Часть 1	Лек	-	0,294
		2	3	Сложные запросы. Работа с представлениями. Продвинутые методы работы с SQL. Часть 2	Лек	-	0,294
		4	1	Подзапросы. CTE. Расширенное понятие о представлениях	Пр	2	0,59
		4	2-3	Оконные функции. Продвинутые методы работы с SQL	Пр	3	0,59
5	Транзакции	2	5	Транзакции	Лек	-	0,294
		4	3-4	Изучение транзакций	Пр	2	0,59
6	Проектирование баз данных	2	7	Проектирование баз данных. Часть 1	Лек	-	0,294
		4	5	Концептуальное проектирование	Пр	2	0,59
		4	6-7	Переход к <u>дatalogической</u> модели данных	Пр	1.5	0,59
		4	7-8	Переход к <u>физической</u> модели данных	Пр	1.5	0,59
		2	9	Самостоятельная работа	Пр	3	0,294
Всего					-	15	5



Календарный план дисциплины «Системы управления базами данных»

**2 семестр, поток ПМ23-1, ПМ23-2, ПМ23-3, ПМ23-4, ПМ23-5, ПМ23-6,
ИД23-1, ИД23-2, ИД23-3, ПИ23-1, ПИ23-2**

Итого за 1 модуль					-	20	-
№	Название темы	Количество часов	№ Недели	Название занятия	Вид	Количество баллов	Посещения
		Модуль 2					
6	Проектирование баз данных (продолжение)	2	9	Проектирование баз данных. Часть 2	Лек	-	0,294
7	Программирование на стороне сервера базы данных (БД), приложения	2	11	Программирование на стороне сервера БД, приложения. Часть 1	Лек	-	0,294
		2	13	Программирование на стороне сервера БД, приложения. Часть 2	Лек	-	0,294
		4	9-10	Функции и процедуры. Язык PL/pgSQL	Пр	2	0,59
		4	11	Триггеры	Пр	2	0,59
		4	12-13	Работа с PostgreSQL в Python	Пр	2	0,59
		2	13	Контрольная работа	Пр	3	0,294
8	Оптимизация выполнения запросов	1	15	Введение в оптимизацию запросов в PostgreSQL	Лек	-	0,147
		6	14-15	Индексы. Индексные структуры. Алгоритмы доступа к данным	Пр	2	0,88
		4	15-16	Способы соединения наборов строк. Подходы к оптимизации запросов	Пр	2	0,59
9	Нереляционные и распределенные СУБД	1	15	Введение в нереляционные и распределенные СУБД	Лек	-	0,147
		2	17	СУБД NoSQL. Распределенные СУБД	Пр	2	0,294
Всего					-	15	5
Итого за 2 модуль					-	20	-



Балльно-рейтинговая система дисциплины "Системы управления базами данных"

Поток ТРПО23-1, ТРПО23-2, ТРПО23-3, ТРПО23-4

№ п/п	Вид отчетности	Баллы
1.	Работа в семестре – всего, в т.ч.:	40
1.1	Работа в первом модуле – всего, в т.ч.	20
1.1.1	Выполнение и защита компьютерных заданий	1,5-4 12
1.1.2	Выполнение самостоятельной работы	3 3
1.1.3	Посещения	0,42 5
1.2	Работа во втором модуле– всего, в т.ч.	20
1.2.1	Выполнение и защита компьютерных заданий	2-3 12
1.2.2	Выполнение контрольной работы	3 3
1.2.3	Посещения	0,38 5
2.	Экзамен	60
	Итого:	100



Календарный план дисциплины «Системы управления базами данных»

2 семестр, поток ТРПО23-1, ТРПО23-2, ТРПО23-3, ТРПО23-4

№	Название темы	Количество часов	№ Недели	Название занятия	Вид	Количество баллов	Посещения
		Модуль 1 (8 семинаров + 4 лекции)					
4	Формирование наборов данных: введение в оператор SELECT (продолжение)	2	1	Сложные запросы. Работа с представлениями. Продвинутые методы работы с SQL. Часть 1	Лек	-	0,42
		2	3	Сложные запросы. Работа с представлениями. Продвинутые методы работы с SQL. Часть 2	Лек	-	0,42
		4	1-2	Подзапросы. СТЕ. Расширенное понятие о представлениях. Оконные функции. Продвинутые методы работы с SQL	Пр	4	0,84
5	Транзакции	2	5	Транзакции	Лек	-	0,42
		2	5	Изучение транзакций	Пр	3	0,42
6	Проектирование баз данных	2	7	Проектирование баз данных. Часть 1	Лек	-	0,42
		4	4-5	Концептуальное проектирование.	Пр	2	0,84
		4	6-7	Переход к <u>дatalogической</u> модели данных	Пр	1.5	0,84
		2	8	Переход к <u>физической</u> модели данных	Пр	1.5	0,42
		-	8	Домашняя самостоятельная работа	-	3	-
Всего					-	15	5
Итого за 1 модуль					-	20	-



Календарный план дисциплины «Системы управления базами данных»

2 семестр, поток ТРПО23-1, ТРПО23-2, ТРПО23-3, ТРПО23-4

№	Название темы	Количество часов	№ Недели	Название занятия	Вид	Количество баллов	Посещения
		Модуль 2 (9 семинаров + 4 лекции)					
6	Проектирование баз данных (продолжение)	2	9	Проектирование баз данных. Часть 2	Лек	-	0,38
7	Программирование на стороне сервера базы данных (БД), приложения	2	11	Программирование на стороне сервера БД, приложения. Часть 1	Лек	-	0,38
		2	13	Программирование на стороне сервера БД, приложения. Часть 2	Лек	-	0,38
		4	9-10	Функции и процедуры. Язык PL/pgSQL	Пр	2	0,76
		4	11-12	Триггеры. Работа с PostgreSQL в Python	Пр	2	0,76
		2	13	Контрольная работа	Пр	3	0,38
8	Оптимизация выполнения запросов	1	15	Введение в оптимизацию запросов в PostgreSQL	Лек	-	0,19
		4	14-15	Индексы. Индексные структуры. Алгоритмы доступа к данным	Пр	2	0,76
		2	16	Способы соединения наборов строк. Подходы к оптимизации запросов	Пр	3	0,38
9	Нереляционные и распределенные СУБД	1	15	Введение в нереляционные и распределенные СУБД	Лек	-	0,19
		2	17	СУБД NoSQL. Распределенные СУБД	Пр	3	0,38
Всего					-	15	5
Итого за 2 модуль					-	20	-



Подзапросы

Подзапросы - мощное средство SQL, они представляют собой запросы, которые встроены в другие запросы, т.е. подзапрос - это запрос в запросе (вложенный запрос).

Когда подзапрос используется во фразе FROM, ему должен быть сопоставлен псевдоним:

```
SELECT * FROM (SELECT aircraft_code, count(*) c
                FROM flights
                GROUP BY aircraft_code) subq
WHERE c > 20;
```

	aircraft_code character	c bigint
1	CN1	9273
2	CR2	9048
3	773	610
4	763	1221
5	319	1239
6	733	1274
7	321	1952
8	SU9	8504



Подзапросы: пример подзапроса в выражении WHERE

```
SELECT count( * )  
  
FROM bookings  
  
WHERE total_amount > ( SELECT AVG( total_amount )  
  
                        FROM bookings );
```

Что подсчитывается? Включите ответ в результаты выполнения работы.



Использование подзапросов в выражении FROM

Подзапрос представляет собой выражение SELECT, которое подставляется в выражение FROM другого оператора SELECT:

```
SELECT * FROM table1 t1
```

```
INNER JOIN
```

```
(SELECT *
```

```
FROM table2 t2
```

```
WHERE t2.attr = 'value') AS t3
```

```
ON t1.t1_id = t3.t1_id
```



Использование подзапросов в списке колонок SELECT

Скалярный вложенный запрос может использоваться в списке колонок выражения SELECT:

```
SELECT t1.t1_name,  
  
(SELECT t2.t2_sum FROM table2 t2  
  
WHERE t2.t1_id = t1.t1_id  
  
AND t2.attr = 'value') AS t1_allsum  
  
FROM table1 t1  
  
ORDER BY t1.t1_id
```



Использование подзапросов в списке колонок SELECT

Скалярный вложенный запрос может использоваться в списке колонок выражения SELECT:

```
SELECT f.flight_id, f.aircraft_code,  
  
       f.scheduled_departure,  
  
       ( SELECT a.airport_name ->> 'ru'  
  
         FROM airports_data a  
  
         WHERE f.departure_airport = a.airport_code)  
  
       AS airport_name  
  
FROM flights f
```

Что выдает запрос? Включите ответ в результаты выполнения работы.

	flight_id [PK] integer	aircraft_code character	scheduled_departure timestamp with time zone	airport_name text
1	1185	319	2017-09-10 09:50:00+03	Домодедово
2	3979	CR2	2017-08-25 14:50:00+03	Внуково
3	4739	763	2017-09-05 12:30:00+03	Внуково
4	5502	763	2017-09-12 09:50:00+03	Шереметьево
5	6938	SU9	2017-09-04 12:25:00+03	Шереметьево
6	7784	CR2	2017-09-10 15:00:00+03	Шереметьево
7	9478	CR2	2017-08-28 09:00:00+03	Пулково
8	11085	733	2017-08-24 15:05:00+03	Кольцово
9	11847	319	2017-09-12 10:15:00+03	Казань
10	12012	CR2	2017-08-26 16:05:00+03	Казань
11	13113	CN1	2017-08-18 16:25:00+03	Ростов-на-Дону
12	14806	CR2	2017-09-06 07:05:00+03	Пермь
13	16837	CN1	2017-09-05 12:25:00+03	Йошкар-Ола
14	17173	CN1	2017-09-14 12:25:00+03	Сыктывкар
15	19807	CN1	2017-09-11 06:35:00+03	Мирный
16	23609	CR2	2017-08-31 11:35:00+03	Бугульма
17	23695	CR2	2017-08-26 10:55:00+03	Бугульма
18	23780	CN1	2017-09-02 06:50:00+03	Стрежевой
19	23945	CR2	2017-09-05 09:15:00+03	Белоярский

Виды подзапросов

- Скалярный подзапрос - выбирает одно значение
- Строковый подзапрос - выбирает одну строку данных
- Подзапрос по столбцу - выбирает один столбец данных
- Табличный подзапрос - выбирает несколько строк



Скалярный подзапрос (повторение)

Скалярный подзапрос должен возвращать строго одно значение!

1) SELECT t.a, t.b

FROM t

WHERE t.c = (SELECT q.q_id

FROM q

WHERE q.q_name = 'test')

2) SELECT t.a, t.b

FROM t

WHERE t.c = (SELECT q.q_id

FROM q

WHERE q.q_name = 'test' LIMIT 1)



Выражения подзапросов

В запросах можно использовать следующие выражения подзапросов:

- EXISTS
- IN
- NOT IN
- ANY/SOME
- ALL
- Сравнение единичных строк



Выражения подзапросов: EXISTS

EXISTS (подзапрос)

Аргументом EXISTS является обычный оператор SELECT, т. е. подзапрос.

Выполнив запрос, система проверяет, возвращает ли он строки в результате. Если он возвращает минимум одну строку, результатом EXISTS будет «true», а если не возвращает ни одной — «false».

Подзапрос может обращаться к переменным внешнего запроса, которые в рамках одного вычисления подзапроса считаются константами.

Подзапрос может выполняться не полностью, а завершаться, как только будет возвращена хотя бы одна строка. Поэтому в подзапросах следует избегать побочных эффектов (например, обращений к генераторам последовательностей).

Так как результат EXISTS зависит только от того, возвращаются строки или нет, но не от их содержимого, список выходных значений подзапроса обычно не имеет значения. Как следствие, широко распространена практика, когда проверки EXISTS записываются в форме EXISTS(SELECT 1 WHERE ...).

Выбрать всех покупателей, которые делали заказы

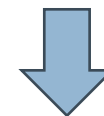
```
SELECT *  
FROM customers c  
WHERE EXISTS (SELECT 1 FROM orders o  
              WHERE o.c_id = c.id)
```

customers

id	name
1	FIO1
2	FIO2
3	FIO3
4	FIO4

orders

c_id	order	odate
2	ord1	12.07
5	ord2	17.08
14	ord3	17.09
4	ord24	22.09



id	name
2	FIO2
4	FIO4



Выражения подзапросов: EXISTS

Этот пример похож на внутреннее соединение по столбцу col2, но он выдаёт максимум одну строку для каждой строки в tab1, даже если в tab2 ей соответствуют несколько строк:

```
SELECT col1
```

```
FROM tab1
```

```
WHERE EXISTS (SELECT 1 FROM tab2 WHERE col2 = tab1.col2);
```



Пример

Получить список книг, которые не продаются ни в одном из магазинов

```
SELECT t1.title_id, t1.title
```

```
FROM titles t1
```

```
WHERE NOT EXISTS
```

```
(SELECT *
```

```
FROM titles t2 INNER JOIN sales s ON
```

```
t2.title_id = s.title_id
```

```
WHERE t2.title_id = t1.title_id)
```



Пример

В какие города нет рейсов из Москвы?

```
SELECT DISTINCT a.city          -- требуется DISTINCT
```

```
FROM airports a
```

```
WHERE NOT EXISTS ( SELECT * FROM routes r
```

```
    WHERE r.departure_city = 'Москва' AND r.arrival_city = a.city)
```



Что такое «Конструктор табличной строки»?

Конструктор табличной строки — это выражение, создающее строку или кортеж (или составное значение) из значений его аргументов-полей.

Конструктор строки состоит из ключевого слова **ROW**, открывающейся круглой скобки, нуля или нескольких выражений (разделённых запятыми), определяющих значения полей, и закрывающейся скобки:

```
SELECT ROW(1, 2.5, 'this is a test'); -- «собирает» строку из трех элементов
```

Если в списке более одного выражения, ключевое слово ROW можно опустить.



Выражения подзапросов: IN

1) Выражение (или конструктор строки) IN (подзапрос)

Если слева - выражение, то справа от IN в скобках задаётся подзапрос, который должен возвращать ровно один столбец.

Если слева - конструктор строки, то справа после IN в скобках записывается подзапрос, который должен вернуть ровно столько столбцов, сколько содержит строка в выражении слева (здесь и далее - для ANY, SOME и т.д.).

Вычисленное значение левого выражения сравнивается со значениями во всех строках, возвращённых подзапросом. Результатом всего выражения IN будет «true», если строка с таким значением находится, и «false» в противном случае (в том числе, когда подзапрос вообще не возвращает строк).

Если результатом выражения слева оказывается NULL или равных значений справа не находится, а хотя бы одно из значений справа равно NULL, конструкция IN возвращает NULL, а не false. Это соответствует принятым в SQL правилам сравнения переменных со значениями NULL. Так же, как и с EXISTS, не следует рассчитывать на то, что подзапрос будет всегда выполняться полностью (здесь и далее - для ANY, SOME и т.д.).

Выбрать всех покупателей, которые купили товары в определенный день:

```
SELECT *  
FROM customers c  
WHERE c.id IN (SELECT c_id FROM orders WHERE  
odate = '12.07')
```

customers

id	name
1	FIO1
2	FIO2
3	FIO3
4	FIO4

orders

c_id	order	odate
2	ord1	12.07
5	ord2	17.08
3	ord3	12.07
4	ord24	22.09

результат

id	name
2	FIO2
3	FIO3



Пример

Список книг, имя автора которых начинается с «А»

```
SELECT t.title_id, t.title
```

```
FROM titles t INNER JOIN title_author ta ON
```

```
t.title_id=ta.title_id
```

```
WHERE ta.au_id IN
```

```
(SELECT a.au_id FROM authors a WHERE
```

```
a.au_fname LIKE 'a%')
```



Пример

Какие маршруты существуют между городами часового пояса Asia/Krasnoyarsk?

```
SELECT flight_no, departure_city, arrival_city
```

```
FROM routes
```

```
WHERE departure_city
```

```
IN ( SELECT city FROM airports WHERE timezone ~ 'Krasnoyarsk' )
```

```
AND arrival_city IN ( SELECT city FROM airports WHERE timezone ~ 'Krasnoyarsk' );
```



2) конструктор_строки IN (подзапрос)

Выражения подзапросов: IN

Выбрать самолеты и места/условия в них,
которые использовались в полетах с
номерами 1 и 2:

```
SELECT * FROM seats
```

```
NATURAL JOIN aircrafts_data
```

```
WHERE
```

```
ROW (seat_no, fare_conditions) IN (
```

```
SELECT seat_no, fare_conditions
```

```
FROM boarding_passes
```

```
WHERE flight_id < 3)
```

	aircraft_code character	seat_no character varying (4)	fare_conditions character varying (10)	model jsonb	range integer
1	773	31F	Economy	{"en": "Boeing 777-300", "ru": "Боинг 777-300"}	11100
2	773	31E	Economy	{"en": "Boeing 777-300", "ru": "Боинг 777-300"}	11100
3	773	31C	Economy	{"en": "Boeing 777-300", "ru": "Боинг 777-300"}	11100
4	773	31B	Economy	{"en": "Boeing 777-300", "ru": "Боинг 777-300"}	11100
5	773	31A	Economy	{"en": "Boeing 777-300", "ru": "Боинг 777-300"}	11100
6	773	30F	Economy	{"en": "Boeing 777-300", "ru": "Боинг 777-300"}	11100
7	773	30E	Economy	{"en": "Boeing 777-300", "ru": "Боинг 777-300"}	11100
8	773	30D	Economy	{"en": "Boeing 777-300", "ru": "Боинг 777-300"}	11100
9	773	30C	Economy	{"en": "Boeing 777-300", "ru": "Боинг 777-300"}	11100
10	773	30B	Economy	{"en": "Boeing 777-300", "ru": "Боинг 777-300"}	11100
11	773	29F	Economy	{"en": "Boeing 777-300", "ru": "Боинг 777-300"}	11100
12	773	29E	Economy	{"en": "Boeing 777-300", "ru": "Боинг 777-300"}	11100
13	773	29D	Economy	{"en": "Boeing 777-300", "ru": "Боинг 777-300"}	11100
14	773	29C	Economy	{"en": "Boeing 777-300", "ru": "Боинг 777-300"}	11100
15	773	29A	Economy	{"en": "Boeing 777-300", "ru": "Боинг 777-300"}	11100

Выражения подзапросов: NOT IN

1) выражение NOT IN (подзапрос)

Справа в скобках записывается подзапрос, который должен возвращать ровно один столбец. Вычисленное значение левого выражения сравнивается со значением во всех строках, возвращённых подзапросом.

Результатом всего выражения NOT IN будет «true», если находятся только несовпадающие строки (в том числе, когда подзапрос вообще не возвращает строк).

Если же находится хотя бы одна подходящая строка, результатом будет «false».

Если результатом выражения слева оказывается NULL или равных значений справа не находится, а хотя бы одно из значений справа равно NULL, конструкция NOT IN возвращает NULL, а не true. Это соответствует принятым в SQL правилам сравнения переменных со значениями NULL.

Не следует рассчитывать на то, что подзапрос будет всегда выполняться полностью.

Выбрать всех покупателей, которые не купили товары в заданный день:

```
SELECT *  
FROM customers c  
WHERE c.id NOT IN (SELECT c_id FROM orders  
WHERE odate = '12.07')
```

customers

id	name
1	FIO1
2	FIO2
3	FIO3
4	FIO4

orders

c_id	order	odate
2	ord1	12.07
1	ord2	17.08
3	ord3	12.07
4	ord24	22.09

результат

id	name
1	FIO1
4	FIO4



Выражения подзапросов: NOT IN

2) конструктор_строки NOT IN (подзапрос)

В левой части этой формы NOT IN записывается конструктор строки. Справа в скобках записывается подзапрос, который должен вернуть ровно столько столбцов, сколько содержит строка в выражении слева.

Вычисленные значения левого выражения сравниваются построчно со значениями во всех строках, возвращённых подзапросом.

Результатом всего выражения NOT IN будет «true», если равных строк не найдётся (в том числе, и когда подзапрос не возвращает строк), и «false», если такие строки есть.

Значения NULL в строках обрабатываются по принятым правилам сравнения (см. ранее).

Если в результатах сравнения строк нет ни одного положительного, но есть хотя бы один NULL, результатом NOT IN будет NULL.

Выбрать самолеты и места/условия в них, которые не использовались в полетах с номерами 1 и 2:

```
SELECT * FROM seats
```

```
NATURAL JOIN aircrafts_data
```

```
WHERE
```

```
    ROW (seat_no, fare_conditions) NOT IN (
```

```
SELECT seat_no, fare_conditions
```

```
FROM boarding_passes
```

```
WHERE flight_id < 3)
```



Выражения подзапросов: ANY/SOME/ALL

Выражение (или конструктор строки) оператор ANY (подзапрос)

Выражение (или конструктор строки) оператор SOME (подзапрос)

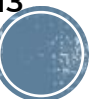
Выражение (или конструктор строки) оператор ALL (подзапрос)

В правой части в скобках записывается подзапрос, который должен возвращать ровно один столбец, если слева - выражение. Если слева - конструктор строки, то подзапрос справа должен вернуть набор строк с таким же набором столбцов (количество, порядок, типы - чтобы можно было сравнить левую и правую части), который создал конструктор строки.

Вычисленное значение левого выражения сравнивается со значением в каждой строке результата подзапроса с помощью заданного оператора условия, который должен выдавать логическое значение. Результатом ANY будет «true», если хотя бы для одной строки условие истинно, и «false» в противном случае (в том числе, и когда подзапрос не возвращает строк). Ключевое слово SOME является синонимом ANY. Конструкцию IN можно также записать как = ANY.

Если условие не выполняется ни для одной из строк, а хотя бы для одной строки условный оператор выдаёт NULL, конструкция ANY возвращает NULL, а не false.

Результатом ALL будет «true», если условие истинно для всех строк (и когда подзапрос не возвращает строк), или «false», если находятся строки, для которых оно ложно. Результатом выражения будет NULL, если ни для одной из строк подзапроса результат сравнения не равен true, а минимум для одной равен NULL. Конструкция NOT IN равнозначна <> ALL.



Пример: ANY

Выбрать пассажиров, которые начали путешествие до 18.07.2017 г.:

```
SELECT passenger_id, passenger_name
FROM tickets
NATURAL JOIN ticket_flights tf
WHERE flight_id = ANY (
    SELECT flight_id
    FROM flights
    WHERE scheduled_departure::DATE < '2017-07-18')
```

	passenger_id character varying (20) 🔒	passenger_name text 🔒
1	6754 366923	ANDREY YAKOVLEV
2	9862 946810	NINA KALININA
3	8878 673564	VIKTOR GAVRILOV
4	4262 169374	MATVEY FEDOTOV
5	5933 228339	ANDREY MAKAROV
6	7200 541610	ALEKSANDR MAKSIMOV
7	8146 986634	SERGEY KOZLOV
8	5439 013617	ANDREY EGOROV
9	3264 296353	OLGA MASLOVA
10	7986 254007	TATYANA NIKITINA
11	6797 970398	MARIYA POTAPOVA
12	8239 551980	ANTONINA KUZNECOVA
13	6486 723999	ARTEM KUZMIN
14	8555 814530	TATYANA NIKOLAEVA
15	7214 511773	YULIYA MOROZOVA
16	3782 676142	RAMIL MOROZOV
17	8690 801470	ANASTASIYA ZHURAVLEVA
18	7905 076349	ELENA GAVRILOVA



Примеры (ANY, SOME):

```
SELECT t.a, t.b FROM t  
WHERE t.c > ANY (SELECT q.size FROM q  
                  WHERE q.q_name LIKE 'test%')
```

```
SELECT t.a, t.b FROM t  
WHERE t.c > SOME (SELECT q.size FROM q  
                  WHERE q.q_name LIKE 'test%')
```



Сравнение единичных строк

конструктор_строки оператор (подзапрос)

Справа в скобках записывается подзапрос, который должен возвращать ровно столько столбцов, сколько содержит строка в выражении слева.

Более того, подзапрос может вернуть максимум одну строку. (Если он не вернёт строк, результатом будет NULL.)

Конструкция возвращает результат сравнения строки слева с этой одной строкой результата подзапроса.

Посмотреть, имеет ли конкретное место (2A) в конкретном самолете (319) класс Business?

```
SELECT ROW ('319', '2A', 'Business') = (  
  SELECT * FROM seats  
  WHERE aircraft_code = '319' and seat_no = '2A')
```

	?column? boolean
1	true



Общие табличные выражения (СТЕ)

В языке SQL есть средства для *последовательного выполнения* и для *повторного использования кода*.

Оператор WITH предоставляет возможность писать вспомогательные операторы для использования в более крупных запросах. Общие табличные выражения (common table expressions - СТЕ) позволяют один раз определить подзапрос и присвоить ему имя, а затем использовать его в нескольких местах главного запроса.

Упрощенный синтаксис СТЕ:

WITH <имя подзапроса> AS (<код подзапроса>) [, ...]

SELECT <список Select> FROM <имя подзапроса>;

СТЕ

Главный
запрос

Здесь код подзапроса - запрос, результаты которого будут впоследствии использованы в главном запросе, как будто это физическая таблица.

Подзапрос в скобках после ключевого слова AS и есть общее табличное выражение. Его можно также назвать подкомандой или вспомогательной командой. После блока WITH находится главный запрос. Команда в целом называется WITH-запросом.



CTE

В CTE можно использовать не только команды SELECT, но также INSERT, UPDATE, и DELETE. В одном WITH-запросе может быть несколько CTE.

Каждому CTE сопоставляется **имя, расположенное перед ключевым словом AS**. Главный запрос может ссылаться на CTE по этому имени.

Одно CTE может ссылаться на другое по имени. Однако разрешается ссылаться только на CTE, определенные раньше.



CTE, пример

```
WITH regional_sales AS ( --первое CTE
    SELECT region, SUM(amount) AS total_sales
    FROM orders
    GROUP BY region
), top_regions AS (      -- второе CTE
    SELECT region
    FROM regional_sales
    WHERE total_sales > (SELECT SUM(total_sales)/10 FROM regional_sales)
)
SELECT region,          --основной запрос
    product,
    SUM(quantity) AS product_units,
    SUM(amount) AS product_sales
FROM orders
WHERE region IN (SELECT region FROM top_regions)
GROUP BY region, product;
```

Смысл SELECT в операторе With — разбить сложные запросы на более простые части

Данный запрос выводит суммы продаж по продуктам только в регионах с наибольшим объемом продаж.

Предложение With определяет два вспомогательных оператора с именами `regional_sales` и `top_regions`, где выходные данные `regional_sales` используются в `top_regions`, а выходные данные `top_regions` используются в основном запросе `SELECT`.



СТЕ, пример

Пусть требуется найти в базе модели машин, выпущенные после 2020 года, и вычислить для них минимальное количество прежних владельцев. Рассмотрим такой код

```
WITH pre_select AS
(
  SELECT car_id, number_of_owners, car_model_id
  FROM car WHERE manufacture_year >= 2020
),
joined_data AS
(
  SELECT car_id, make, model, number_of_owners
  FROM pre_select
  INNER JOIN car_model ON pre_select.car_model_id= car_model.car_model_id
),
minimal_owners AS (SELECT min(number_of_owners) AS min_number_of_owners
FROM pre_select)
SELECT car_id, make, model, number_of_owners
FROM joined_data INNER JOIN minimal_owners
ON joined_data.number_of_owners = minimal_owners.min_number_of_owners;
```

Логическая часть запроса представлена в виде последовательности действий: фильтрация в pre_select и соединение в joined_data.

Вторая часть - вычисление количества владельцев - выполняется в отдельном подзапросе, minimal_owners.

Что выбирается с помощью pre_select, что соединяется в joined_data? Включите ответы в результаты выполнения работы.



Рекурсивные и иерархические запросы

CTE может ссылаться на себя же. Такие команды называются рекурсивными запросами. У этих запросов своеобразная структура, сообщающая базе данных об их характере:

```
WITH RECURSIVE <имя_подзапроса> (<список полей>) AS  
(  
  <нерекурсивная часть> ← якорный подзапрос  
  UNION [ALL | DISTINCT]  
  <рекурсивная часть> ← итеративный подзапрос  
)  
[,...]  
<главный запрос>
```

Обе части - нерекурсивная и рекурсивная - подзапросы, которые должны возвращать одинаковое число полей одинаковых типов. Имена полей задаются в объявлении рекурсивного запроса в целом, поэтому не важно, как они названы в подзапросах.



Рекурсивные и иерархические запросы

Якорный подзапрос (нерекурсивная часть) - отправная точка для выполнения рекурсивного запроса. Он не может ссылаться на имя рекурсивного запроса и выполняется только один раз.

Результаты якорного подзапроса передаются назад тому же самому СТЕ, после чего выполняется рекурсивная часть.

Такие итерации повторяются до тех пор, пока результат рекурсивного подзапроса не окажется пустым.

Результатом всего запроса является объединение множества строк, возвращенных нерекурсивной частью и всеми итерациями рекурсивной части.

Если присутствует фраза UNION ALL, то возвращаются все эти строки, а если UNION DISTINCT или просто UNION, то дубликаты удаляются.



Рекурсивные и иерархические запросы

Вычисление рекурсивного запроса (алгоритм):

1. Вычисляется нерекурсивная часть. Для UNION (но не UNION ALL) отбрасываются дублирующиеся строки. Все оставшиеся строки включаются в результат рекурсивного запроса и также помещаются во временную рабочую таблицу.
2. Пока рабочая таблица не пуста, повторяются следующие действия:
 - А) Вычисляется рекурсивная часть так, что рекурсивная ссылка на сам запрос обращается к текущему содержимому рабочей таблицы. Для UNION (но не UNION ALL) отбрасываются дублирующиеся строки и строки, дублирующие ранее полученные. Все оставшиеся строки включаются в результат рекурсивного запроса и также помещаются во временную промежуточную таблицу.
 - Б) Содержимое рабочей таблицы заменяется содержимым промежуточной таблицы, а затем промежуточная таблица очищается.



Рекурсивные и иерархические запросы

Алгоритм выполнения рекурсивного запроса подразумевает итерацию, а не рекурсию. Однако в стандарте SQL такие запросы называются рекурсивными. В других базах данных та же логика реализуется аналогично, синтаксис может отличаться.

Пример: рекурсивный запрос для вычисления факториала:

```
WITH RECURSIVE subq (n, factorial) AS  
(  
  SELECT 1, 1  
  UNION ALL  
  SELECT n + 1, factorial * (n + 1) from subq WHERE n < 5  
)  
SELECT * FROM subq;
```

n		factorial
1		1
2		2
3		6
4		24
5		120
(5 rows)		



Рекурсивные и иерархические запросы

В примере `SELECT 1, 1` - якорный подзапрос. Он возвращает одну строку (оба поля `n` и `factorial` равны 1), которая передается следующей итерации.

На первой итерации к значению `n` прибавляется единица, и значение `factorial` умножается на $(n+1)$, и получается 2 и 2.

Эта строка передается следующей итерации, на которой вычисляются значения 3 и 6. И так далее. На последней итерации возвращается строка, в которой поле `n` равно 5.

Эта строка отфильтровывается фразой `WHERE` итеративного подзапроса, поэтому на следующей итерации не возвращается ничего, и выполнение останавливается.

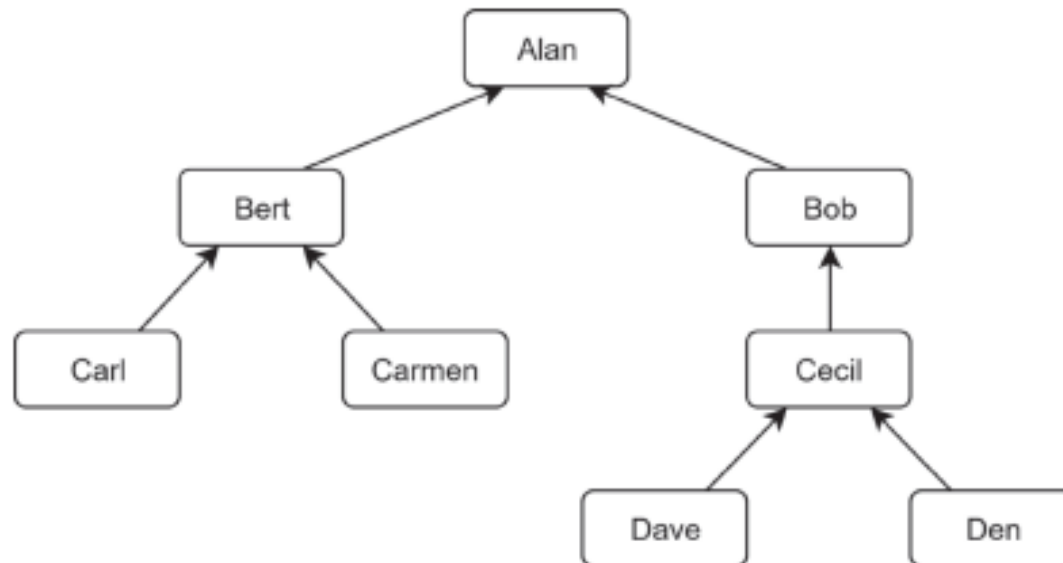
Весь рекурсивный запрос возвращает список из пяти чисел от 1 до 5 и соответствующие им факториалы.



Рекурсивные и иерархические запросы

Обычно иерархия подразумевает наличие связи «родитель-потомок» между объектами, когда один и тот же объект может одновременно быть родителем и потомком.

Рассмотрим семью. У Алана два ребенка: Берт и Боб. У Берта тоже два ребенка: Карл и Кармен. У Боба один ребенок, Сесиль, у которой двое детей: Дэйв и Дэн. Генеалогическое древо показано на рисунке ниже. Стрелки направлены от ребенка к родителю.



Рекурсивные и иерархические запросы

В базе данных сведения об иерархии можно хранить в обычной таблице с двумя столбцами: id и link.

Первый столбец (id, хранит имя персоны) будет первичным ключом, второй (link, хранит имя родителя персоны) - внешним ключом, ссылающимся на ту же таблицу:

```
CREATE TABLE family (
```

```
    id text PRIMARY KEY,
```

```
    link text REFERENCES family
```

```
);
```

```
INSERT INTO family VALUES ('Alan', NULL),
```

```
('Bert', 'Alan'), ('Bob', 'Alan'), ('Carl', 'Bert'), ('Carmen', 'Bert'),
```

```
('Cecil', 'Bob'), ('Dave', 'Cecil'), ('Den', 'Cecil');
```

```
INSERT 0 8
```

В первой вставленной записи поле link равно NULL, это означает, что о родителе Алана нет информации



Рекурсивные и иерархические запросы

Пусть требуется построить полные родословные всех детей. Сделать это путем соединения таблиц невозможно, потому что каждое соединение описывает только один уровень иерархии, а в общем случае количество уровней неизвестно.

Задачу решает следующий рекурсивный запрос:

```
WITH RECURSIVE genealogy (bloodline, id, level) AS
(
  SELECT id, id, 0 FROM family WHERE link IS NULL
  UNION ALL
  SELECT g.bloodline || ' -> ' || f.id, f.id, g.level + 1
  FROM family f, genealogy g WHERE f.link = g.id
)
SELECT bloodline, level FROM genealogy;
```

Добавьте в генеалогическое древо трех детей Кармен и двух детей Дэна. Выполните вставку соответствующих данных в таблицу и запрос. Полученные результаты добавьте в результаты выполнения работы.

bloodline	level
Alan	0
Alan -> Bert	1
Alan -> Bob	1
Alan -> Bert -> Carl	2
Alan -> Bert -> Carmen	2
Alan -> Bob -> Cecil	2
Alan -> Bob -> Cecil -> Dave	3
Alan -> Bob -> Cecil -> Den	3

(8 rows)



Интерпретация

В нерекурсивной части выбирается начало иерархии, т. е. человек, не имеющий родителя. Его имя будет стоять в начале каждой родословной.

На первой итерации рекурсивного подзапроса выбираются его дети (те, у которых в поле link находится значение поля id, выбранное нерекурсивным подзапросом). Их имена добавляются в конец родословной через разделитель ->.

На второй итерации выбираются дети детей и т. д. Если детей не найдено, выполнение останавливается. Значение в поле level (уровень) увеличивается на 1 на каждой итерации, чтобы можно было показать номер итерации в результатах.

При выполнении таких запросов может возникнуть проблема. Если в данных имеются циклы, то данный рекурсивный запрос никогда не остановится.

Изменим запись в таблице family:

```
UPDATE family SET link = 'Bert' WHERE id = 'Alan';
```

Теперь в данных имеется цикл: Алан является ребенком собственного ребенка.



Цикл в данных: решение

Если снова запустить запрос для построения родословной, то он будет крутиться в цикле, пока не завершится ошибкой*. Чтобы заставить запрос остановиться, можно проверять, был ли человек, обрабатываемый подзапросом, уже включен в родословную:

```
WITH RECURSIVE genealogy (bloodline, id, level, processed)
AS
(
    SELECT id, id, 0, ARRAY[id] FROM family WHERE id = 'Alan'
    UNION ALL
    SELECT g.bloodline || ' -> ' || f.id, f.id, g.level + 1,
    processed || f.id
    FROM family f, genealogy g
    WHERE f.link = g.id AND NOT f.id = ANY(processed)
)
SELECT bloodline, level FROM genealogy;
```



Интерпретация

Результат тот же.

Поле `processed` используется в CTE, но не выбирается в главном запросе. В нем хранятся те же данные, что в поле `bloodline`, но в такой форме, чтобы их было удобно использовать во фразе `WHERE`.

На каждой итерации в массив добавляется имя только что обработанного человека.

Кроме того, в условии `WHERE` в рекурсивном подзапросе дополнительно проверяется, что обрабатываемый человек отсутствует в массиве `processed`.

На рекурсивные запросы накладываются ограничения: в рекурсивной части не должно быть агрегирования, а на имя рекурсивного запроса разрешается ссылаться только в рекурсивной части.



Рекурсивные и иерархические запросы

Пример: сумма чисел от 1 до 100:

```
WITH RECURSIVE t(n) AS (
```

```
    VALUES (1)
```

```
    UNION ALL
```

```
    SELECT n+1 FROM t WHERE n < 100
```

```
)
```

```
SELECT sum(n) FROM t;
```



Рекурсивные и иерархические запросы

Для тестирования запросов, которые могут зацикливаться, есть хороший приём — добавить LIMIT в родительский запрос. Например, следующий запрос зациклится, если не добавить предложение LIMIT:

```
WITH RECURSIVE t(n) AS (  
    SELECT 1  
    UNION ALL  
    SELECT n+1 FROM t  
)  
SELECT n FROM t LIMIT 50;
```

В данном случае этого не происходит, так как в Postgres запрос WITH выдаёт столько строк, сколько фактически принимает родительский запрос. В производственной среде использовать этот приём не рекомендуется, так как другие системы могут вести себя по-другому.

Кроме того, это не будет работать, если внешний запрос сортирует результаты рекурсивного запроса или соединяет их с другой таблицей, так как в подобных случаях внешний запрос обычно всё равно выбирает результат запроса WITH полностью.



Поиск в глубину. Порядок поиска

При вычислении обхода дерева с помощью рекурсивного запроса можно упорядочить результаты либо в глубину, либо в ширину. Это можно сделать, вычислив столбец упорядочения вместе с другими столбцами данных и используя его для сортировки результатов в конце.

Чтобы выполнить поиск в глубину, мы вычисляем для каждой строки результата массив строк, которые мы уже посетили. Например, рассмотрим следующий запрос (data - случайные данные):

```
WITH RECURSIVE search_tree(id, link, data) AS (  
    SELECT t.id, t.link, t.data  
    FROM family t  
    UNION ALL  
    SELECT t.id, t.link, t.data  
    FROM family t, search_tree st  
    WHERE t.id = st.link  
)  
SELECT * FROM search_tree;
```

	id text	link text	data bigint
1	Alan	[null]	5
2	Bert	Alan	6
3	Bob	Alan	4
4	Carl	Bert	34
5	Carmen	Bert	56
6	Cecil	Bob	67
7	Dave	Cecil	78
8	Den	Cecil	67
9	Alan	[null]	5
10	Alan	[null]	5
11	Bert	Alan	6
12	Bert	Alan	6
13	Bob	Alan	4
14	Cecil	Bob	67
15	Cecil	Bob	67
16	Alan	[null]	5
17	Alan	[null]	5
18	Alan	[null]	5
19	Bob	Alan	4
20	Bob	Alan	4
21	Alan	[null]	5
22	Alan	[null]	5



Поиск в глубину

Чтобы добавить информацию о цепочке узлов в глубину, можно написать следующее (если использовать условие «t.id = st.link», как здесь, обход получается от детей к родителям):



```
WITH RECURSIVE search_tree(id, link, data, path)
AS (
    SELECT t.id, t.link, t.data, ARRAY[t.id]
    FROM family t
    UNION ALL
    SELECT t.id, t.link, t.data, path || t.id
    FROM family t, search_tree st
    WHERE t.id = st.link
)
SELECT * FROM search_tree ORDER BY path;
```

	id text	link text	data bigint	path text[]
1	Alan	[null]	5	{Alan}
2	Bert	Alan	6	{Bert}
3	Alan	[null]	5	{Bert,Alan}
4	Bob	Alan	4	{Bob}
5	Alan	[null]	5	{Bob,Alan}
6	Carl	Bert	34	{Carl}
7	Bert	Alan	6	{Carl,Bert}
8	Alan	[null]	5	{Carl,Bert,Alan}
9	Carmen	Bert	56	{Carmen}
10	Bert	Alan	6	{Carmen,Bert}
11	Alan	[null]	5	{Carmen,Bert,Alan}
12	Cecil	Bob	67	{Cecil}
13	Bob	Alan	4	{Cecil,Bob}
14	Alan	[null]	5	{Cecil,Bob,Alan}
15	Dave	Cecil	78	{Dave}
16	Cecil	Bob	67	{Dave,Cecil}
17	Bob	Alan	4	{Dave,Cecil,Bob}
18	Alan	[null]	5	{Dave,Cecil,Bob,Alan}
19	Den	Cecil	67	{Den}
20	Cecil	Bob	67	{Den,Cecil}
21	Bob	Alan	4	{Den,Cecil,Bob}
22	Alan	[null]	5	{Den,Cecil,Bob,Alan}



Поиска в глубину: пример

Правильно будет выполнять обход от родителей к детям, как это указано на исходном графе. Так как *link* в таблице *family* содержит идентификаторы родительских узлов, а *id* является идентификатором текущего узла, при задании условия *t.link = st.id*, ищем узлы, которые являются дочерними по отношению к текущему узлу в CTE *search_tree*. Сортируем только по *path* и получаем обход в глубину.

Для представленного выше генеалогического древа запрос

```
WITH RECURSIVE search_tree(id, link, path) AS (
```

```
    SELECT t.id, t.link, ARRAY[t.id]
```

```
    FROM family t
```

```
    UNION ALL
```

```
    SELECT t.id, t.link, path || t.id
```

```
    FROM family t, search_tree st
```

```
    WHERE t.link = st.id
```

```
)
```

```
SELECT id, link, path FROM search_tree ORDER BY path;
```

Возвращает следующий результат **поиска в глубину** (показывает цепочку предков для id):

	Data Output	Messages	Notifications
	id text	link text	path text[]
1	Alan	[null]	{Alan}
2	Bert	Alan	{Alan,Bert}
3	Carl	Bert	{Alan,Bert,Carl}
4	Carmen	Bert	{Alan,Bert,Carmen}
5	Bob	Alan	{Alan,Bob}
6	Cecil	Bob	{Alan,Bob,Cecil}
7	Dave	Cecil	{Alan,Bob,Cecil,Dave}
8	Den	Cecil	{Alan,Bob,Cecil,Den}
9	Bert	Alan	{Bert}
10	Carl	Bert	{Bert,Carl}
11	Carmen	Bert	{Bert,Carmen}
12	Bob	Alan	{Bob}
13	Cecil	Bob	{Bob,Cecil}
14	Dave	Cecil	{Bob,Cecil,Dave}
15	Den	Cecil	{Bob,Cecil,Den}
16	Carl	Bert	{Carl}
17	Carmen	Bert	{Carmen}
18	Cecil	Bob	{Cecil}
19	Dave	Cecil	{Cecil,Dave}
20	Den	Cecil	{Cecil,Den}
21	Dave	Cecil	{Dave}
22	Den	Cecil	{Den}



Пример поиска в ширину

Запрос тот же,
за исключением
сортировки.
Для поиска в
ширину
сортируем
результат
вначале по level,
затем - по path.

Для представленного выше генеалогического дерева
запрос

```
WITH RECURSIVE search_tree(id, link, path, level) AS (  
    SELECT t.id, t.link, ARRAY[t.id], 1  
    FROM family t  
    UNION ALL  
    SELECT t.id, t.link, path || t.id, st.level + 1  
    FROM family t  
    JOIN search_tree st ON t.link = st.id  
    WHERE st.level < 5 -- Максимальная глубина  
поиска  
)  
SELECT id, link, path, level  
FROM search_tree  
ORDER BY level, path; Возвращает следующий  
результат поиска в ширину:
```

	id text	link text	path text[]	level integer
1	Alan	[null]	{Alan}	1
2	Bert	Alan	{Bert}	1
3	Bob	Alan	{Bob}	1
4	Carl	Bert	{Carl}	1
5	Carmen	Bert	{Carmen}	1
6	Cecil	Bob	{Cecil}	1
7	Dave	Cecil	{Dave}	1
8	Den	Cecil	{Den}	1
9	Bert	Alan	{Alan,Bert}	2
10	Bob	Alan	{Alan,Bob}	2
11	Carl	Bert	{Bert,Carl}	2
12	Carmen	Bert	{Bert,Carmen}	2
13	Cecil	Bob	{Bob,Cecil}	2
14	Dave	Cecil	{Cecil,Dave}	2
15	Den	Cecil	{Cecil,Den}	2
16	Carl	Bert	{Alan,Bert,Carl}	3
17	Carmen	Bert	{Alan,Bert,Carmen}	3
18	Cecil	Bob	{Alan,Bob,Cecil}	3
19	Dave	Cecil	{Bob,Cecil,Dave}	3
20	Den	Cecil	{Bob,Cecil,Den}	3
21	Dave	Cecil	{Alan,Bob,Cecil,Dave}	4
22	Den	Cecil	{Alan,Bob,Cecil,Den}	4



Изменение данных в WITH

Вложенные операторы в WITH выполняются одновременно друг с другом и с основным запросом. Таким образом, порядок, в котором операторы в WITH будут фактически изменять данные, непредсказуем. Все эти операторы выполняются с одним снимком данных, так что они не могут «видеть», как каждый из них меняет целевые таблицы. Это уменьшает эффект непредсказуемости фактического порядка изменения строк и означает, что RETURNING — единственный вариант передачи изменений от вложенных операторов WITH основному запросу. Например, в данном случае:

```
WITH t AS (  
    UPDATE products SET price = price * 1.05  
    RETURNING *  
)  
SELECT * FROM products;
```

внешний оператор SELECT выдаст цены, которые были до действия UPDATE, тогда как в запросе

```
WITH t AS (  
    UPDATE products SET price = price * 1.05  
    RETURNING *  
)  
SELECT * FROM t;
```

внешний SELECT выдаст изменённые данные.

**Заполните таблицу products небольшим количеством данных, выполните оба запроса, результаты включите в общие результаты выполнения заданий по теме.*



Изменение данных в WITH

В предложении WITH можно также использовать операторы, изменяющие данные (INSERT, UPDATE или DELETE). Это позволяет выполнять в одном запросе сразу несколько разных операций. Например:

```
WITH moved_rows AS (  
    DELETE FROM products  
    WHERE  
        "date" >= '2010-10-01' AND  
        "date" < '2010-11-01'  
    RETURNING *  
)  
INSERT INTO products_log  
SELECT * FROM moved_rows;
```

Этот запрос перемещает строки из products в products_log. Оператор DELETE в WITH удаляет указанные строки из products и возвращает их содержимое в предложении RETURNING; а затем главный запрос читает это содержимое и вставляет в таблицу products_log.

Следует заметить, что предложение WITH в данном случае присоединяется к оператору INSERT, а не к SELECT, вложенному в INSERT.



Представления

Представление можно считать именованным запросом или оберткой вокруг команды SELECT. Используются для следующих целей:

- чтобы упростить сложные запросы и повысить степень модульности кода;
- для повышения производительности посредством кеширования результатов и использования их в будущем;
- для уменьшения объема SQL-кода;
- чтобы перебросить мост между реляционными базами данных и объектно-ориентированными языками (особенно в этом смысле полезны обновляемые представления);
- чтобы реализовать авторизацию на уровне строк - не давать доступа к строкам, не удовлетворяющим заданному условию;
- для реализации интерфейсов и уровня абстракции, расположенного между языками высокого уровня и реляционными базами;
- для реализации срочных изменений.



Представления

Команда CREATE VIEW позволяет создать представление, а при наличии ключевого слова REPLACE заменить уже существующее представление.

Имена атрибутов представления можно задать явно, в противном случае они наследуются от команды SELECT:

```
CREATE [ OR REPLACE ] [ TEMP | TEMPORARY ] [ RECURSIVE ] VIEW name [ (  
column_name [, ...] ) ]  
[ WITH ( view_option_name [= view_option_value] [, ... ] ) ]  
AS query  
[ WITH [ CASCADED | LOCAL ] CHECK OPTION ]
```



Представления

Упрощенный синтаксис команды CREATE VIEW, предназначенной для создания представлений:

CREATE VIEW имя-представления [(имя-столбца [, ...])]

AS запрос;

В этой команде обязательными элементами являются имя представления и запрос к базе данных, который и формирует выборку из нее.

Если список имен столбцов не указан, тогда их имена «вычисляются» (формируются) на основании текста запроса.

В отличие от таблиц, представления не содержат данных. При каждом обращении к представлению в команде SELECT данные выбираются из таблиц, на основе которых это представление создано.



Представления

Ниже создается представление, которое выводит всю информацию о пользователе, кроме пароля (чтобы ограничить доступ приложения к паролю). Имена столбцов наследуются от столбцов в команде SELECT, как показывает метакоманда \d для объекта account_information:

```
CREATE VIEW account_information AS SELECT account_id,  
first_name, last_name, email FROM account;
```

```
CREATE VIEW
```

```
=> \d account_information
```

```
View "car_portal_app.account_information"
```

Column	Type	Collation	Nullable	Default
account_id	integer			
first_name	text			
last_name	text			
email	text			



Представления

Имена столбцов представления можно задать явно:

```
CREATE OR REPLACE VIEW account_information (account_id,first_name,last_name,email)  
AS SELECT account_id, first_name,  
last_name, email FROM account;
```

Если определение представления изменяется с помощью ключевого слова REPLACE, то списки столбцов в старом и новом представлениях должны быть одинаковы, включая имя, тип и порядок следования.

Ниже показано, что произойдет при попытке изменить порядок столбцов:

```
=> CREATE OR REPLACE VIEW account_information AS SELECT  
account_id, last_name, first_name, email FROM account;
```

ERROR: cannot change name of view column "first_name" to "last_name"

Если же вы хотите изменить имя хотя бы одного столбца, то сначала нужно удалить представление с помощью команды DROP VIEW, а уже затем создать его заново.



Представления

Посмотреть описание представления в базе данных можно с помощью команды

```
\d flights_v
```

В представлении «Рейсы» базы данных авиаперелетов много столбцов, поэтому при выводе информации из него в виде таблицы каждая строка на экране будет сворачиваться «змейкой», что не очень наглядно. Утилита `psql` предлагает альтернативный – расширенный – способ вывода информации, который включается с помощью команды

```
\x
```

Для возвращения к табличному формату вывода нужно выполнить эту же команду еще раз.

Включив расширенный вывод, выполните команду для выборки данных из представления «Рейсы». Результаты поместите в результаты выполнений заданий по теме.



Категории представлений

В PostgreSQL имеется несколько категорий представлений.

- Временные представления. Такое представление автоматически удаляется в конце сеанса. Если ключевое слово `TEMPORARY` или `TEMP` отсутствует, то жизненный цикл представления начинается в момент создания и заканчивается в момент явного удаления.
- Рекурсивные представления. Рекурсивное представление напоминает рекурсивную функцию в языках программирования. Список столбцов в этом случае обязателен. Рекурсия, в частности рекурсивные представления и рекурсивные общие табличные выражения (CTE), позволяет строить очень сложные запросы, особенно для иерархически организованных данных.



Категории представлений

- Обновляемые представления. Обновляемые представления позволяют обращаться с представлениями, как с таблицами, т. е. выполнять команды INSERT, UPDATE и DELETE.

Обновляемые представления в какой-то мере можно рассматривать как мост между реляционной и объектной моделями, они дают некий аналог полиморфизма.

- Материализованные представления. Это, по сути дела, таблица, содержимое которой периодически обновляется заранее заданным запросом.

Материализованные представления повышают производительность запросов, которые выполняются долго, и часто применяются к статическим данным. Можно считать их вариантом кеширования.



Материализованные представления

Материализованные представления - расширение PostgreSQL, но их поддерживают и другие СУБД, например Oracle. Материализованное представление хранит и запрос, и данные.

```
CREATE MATERIALIZED VIEW [ IF NOT EXISTS ] table_name
```

```
[ (column_name [, ...] ) ]
```

```
[ WITH ( storage_parameter [= value] [, ... ] ) ]
```

```
[ TABLESPACE tablespace_name ]
```

```
AS query
```

```
[ WITH [ NO ] DATA ]
```



Материализованные представления

В момент создания материализованного представления его можно заполнить или оставить пустым. Для заполнения пустого материализованного представления служит команда REFRESH MATERIALIZED VIEW с таким синтаксисом:

```
REFRESH MATERIALIZED VIEW [ CONCURRENTLY ] name [ WITH [ NO ] DATA ]
```

Попытка выбрать данные из незаполненного представления заканчивается ошибкой:

```
=> CREATE MATERIALIZED VIEW test_mat AS SELECT 1 WITH NO DATA;
```

```
CREATE MATERIALIZED VIEW
```

```
=> TABLE test_mat;
```

```
ERROR: materialized view "test_mat" has not been populated
```

```
HINT: Use the REFRESH MATERIALIZED VIEW command.
```

Пример: REFRESH MATERIALIZED VIEW routes;



Упражнения.

Для выполнения заданий изучите схему и установите демонстрационную базу данных для СУБД PostgreSQL, в которой хранятся данные об авиаперевозках:

<https://postgrespro.ru/education/demodb>

Для установки скачайте один из скриптов, позволяющих развернуть БД, например, [demo-small.zip](#) (21 МБ). Распакуйте архив.

Пусть, например, получился следующий полный путь к файлу со скриптом:

C:\Users\User1\Downloads\demo-small.sql

Запустите SQL Shell (psql) из установленного пакета PostgreSQL. При запуске нажмите 4 раза Enter на приглашения Server, Database, Port, Username, а на приглашение Пароль пользователя Postgres введите пароль (в компьютерных классах пароль postgres) – буквы отображаться не будут (пароль postgres используется в компьютерных классах университета).

Выполните в psql команду (путь задается в апострофах, обратные слэши экранируются: \).

psql \i 'C:\\Users\\User1\\Downloads\\demo-small.sql'

[Возможны другие варианты запуска скрипта. Например, можно следовать инструкции по установке, по-другому писать путь и т.д.]

Для переключения на demo БД авиаперевозок можно использовать команду: \c demo

Посмотрите список таблиц: \dt

Посмотрите список полей для каждой таблицы, например, таблицы flights: \d flights

Запросы можно писать в Query Tool клиента pgAdmin (база данных demo, схема bookings, в ней таблицы Tables и представления View) или в psql.

Если в ответ на запрос, например: **SELECT * FROM flights;**

выдается много строк, можно прервать вывод, нажав Ctrl+C. Можно также ограничить вывод с помощью LIMIT n:

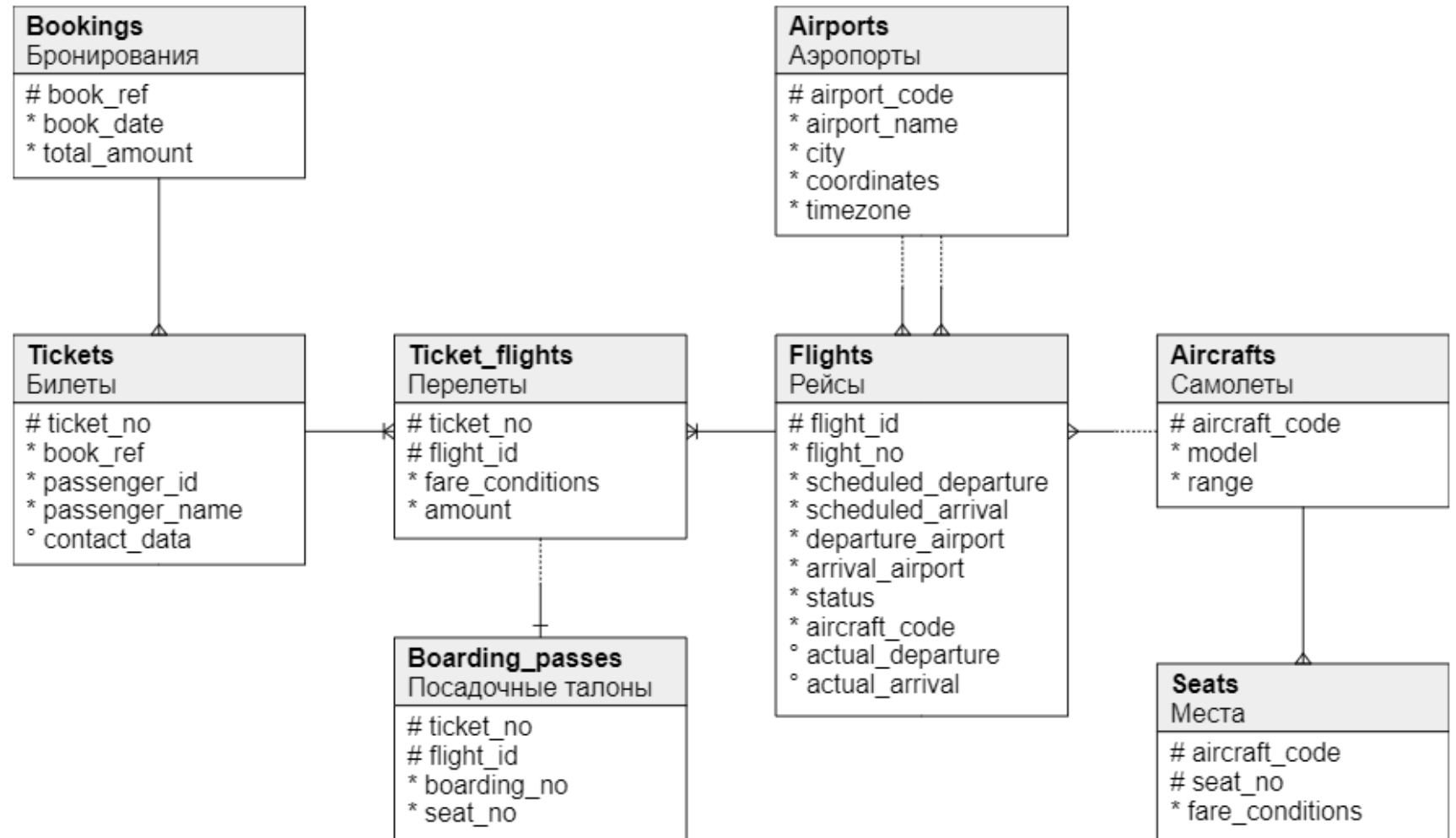
SELECT * FROM flights LIMIT 10; -- вывести первые десять строк



Упражнения.

Диаграмма схемы данных
БД demo (в нотации
Баркера):

<https://postgrespro.ru/docs/postgrespro/10/apjs02>



Упражнения.

Описание схемы: <https://postgrespro.ru/docs/postgrespro/10/apjs03>

Основной сущностью является бронирование (bookings).

В одно бронирование можно включить несколько пассажиров, каждому из которых выписывается отдельный билет (tickets). Билет имеет уникальный номер и содержит информацию о пассажире. Как таковой пассажир не является отдельной сущностью. Как имя, так и номер документа пассажира могут меняться с течением времени, так что невозможно однозначно найти все билеты одного человека; для простоты можно считать, что все пассажиры уникальны.

Билет включает один или несколько перелетов (ticket_flights). Несколько перелетов могут включаться в билет в случаях, когда нет прямого рейса, соединяющего пункты отправления и назначения (полет с пересадками), либо когда билет взят «туда и обратно». В схеме данных нет жёсткого ограничения, но предполагается, что все билеты в одном бронировании имеют одинаковый набор перелетов.

Каждый рейс (flights) следует из одного аэропорта (airports) в другой. Рейсы с одним номером имеют одинаковые пункты вылета и назначения, но будут отличаться датой отправления.

При регистрации на рейс пассажиру выдаётся посадочный талон (boarding_passes), в котором указано место в самолете. Пассажир может зарегистрироваться только на тот рейс, который есть у него в билете. Комбинация рейса и места в самолете должна быть уникальной, чтобы не допустить выдачу двух посадочных талонов на одно место.

Количество мест (seats) в самолете и их распределение по классам обслуживания зависит от модели самолета (aircrafts), выполняющего рейс. Предполагается, что каждая модель самолета имеет только одну компоновку салона. Схема данных не контролирует, что места в посадочных талонах соответствуют имеющимся в самолете.

Подробное описание объектов: <https://postgrespro.ru/docs/postgrespro/10/apjs04>



Упражнения (блок 1 - подзапросы). Номер варианта = (Номер_по_журналу % 10) + 1, % - остаток от деления

Столбец: задача, строка: номер варианта	1. Напишите запрос на основе использования подзапросов, который выявляет направления, на которые было продано:	2. Напишите запрос на основе использования подзапросов, который подсчитывает количество операций бронирования, в которых общая сумма не превышает q от p величины по всей выборке.	3. Напишите запрос на основе использования подзапросов, который выводит для каждой модели самолета количество мест класса business, количество мест класса comfort и количество мест класса есопому, которые занимали более p пассажиров в q-м месяце	4. Напишите запрос на основе использования подзапросов, который позволяет получить перечень аэропортов в тех городах (город, код аэропорта, название аэропорта), в которых больше q аэропортов.	5. Придумайте и напишите по одному запросу для каждого выражения подзапроса из множества w к базе данных об авиаперевозках. Запросы у разных студентов не должны повторяться.
1	0 билетов	$q = 2/3$, p = максимальной	$p = 10$, $q = 7$	$q = 1$	$w = \{EXISTS, NOT\ IN, ANY\}$
2	1 билет	$q = 1/3$, p = максимальной	$p = 5$, $q = 8$	$q = 2$	$w = \{IN, SOME, ALL\}$
3	3 билета	$q = 1/2$, p = средней	$p = 100$, $q = 9$	$1 = 1$	$w = \{NOT\ EXISTS, IN, SOME\}$
4	5 билетов	$q = 0.7$, p = средней	$p = 120$, $q = 7$	$q = 2$	$w = \{EXISTS, IN, ALL\}$
5	более 12 билетов	$q = 0.2$, p = максимальной	$p = 15$, $q = 8$	$q = 0$	$w = \{NOT\ IN, ANY, ALL\}$
6	6 билетов	$q = 0.8$, p = средней	$p = 60$, $q = 9$	$q = 1$	$w = \{NOT\ EXISTS, NOT\ IN, ANY\}$
7	менее 5 билетов	$q = 0.9$, p = средней	$p = 10000$, $q = 7$	$q = 2$	$w = \{EXISTS, IN, ANY\}$
8	от 20 до 500 билетов	$q = 0.45$, p = средней	$p = 50000$, $q = 8$	$q = 1$	$w = \{IN, SOME, ALL\}$
9	8 билетов	$q = 0.68$, p = максимальной	$p = 11$, $q = 9$	$q = 2$	$w = \{EXISTS, ANY, ALL\}$
10	более 100 билетов	$q = 0.9$, p = максимальной	$p = 4500$, $q = 7$	$q = 0$	$w = \{NOT\ EXISTS, SOME, ALL\}$



Упражнения (блок 2 - СТЕ). Номер варианта = (Номер_по_журналу % 10) + 1, % - остаток от деления

Столбец: задача, строка: номер варианта	1. Напишите запрос на основе СТЕ, который ищет в базе перелеты, совершенные после даты р (день.месяц), и выводит по ним информацию о пассажирах, которые летели q-классом.	2. Напишите запрос на основе СТЕ, который ищет максимальное количество пассажиров, совершивших минимальное количество перелетов в месяце q, в рейсах, совершенных из городов, название которые начинается с буквы р.	3. Реализуйте рекурсивный запрос для вычисления значения выражения	4. Выполните с помощью рекурсивных запросов обходы созданного дерева комментариев (как создать - см. далее) в глубину и ширину, сделайте скриншоты запросов и результатов. У студентов результаты этой задачи не должны повторяться. Создайте таблицу, в которой будут храниться комментарии чата, имеющие иерархическую структуру: каждый комментарий может иметь до пяти потомков. Заполните таблицу выдуманными данными, в таблице должно быть:
1	p=28.07, q=Business	q = 7, p = «М»	Сумма шести величин $n \cdot (n + 1)$ при начальном $n_0 = 1$, с шагом 2: $n_1 = 3$ и т.д.	не менее двух комментариев с тремя потомками, не менее 3 комментариев с четырьмя потомками, не менее 4 комментариев с двумя потомками, комментарии с одним потомком, комментарии без потомков (следует хранить не только id комментариев, но и сами комментарии).
2	p=15.07, q=Economy	q = 8, p = «К»	Сумма десяти слагаемых $n / (n + 1)$ при начальном $n_0 = 10$, с шагом 1: $n_1 = 11$ и т.д.	не менее двух комментариев с 2 потомками, не менее 3 комментариев с 3 потомками, не менее 4 комментариев с 4 потомками, комментарии с одним потомком, комментарии без потомков (следует хранить не только id комментариев, но и сами комментарии).
3	p=01.08, q=Comfort	q = 9, p = «С»	Сумма восьми слагаемых $n + n^2 + n^3$ при начальном $n_0 = 2$, с шагом 3: $n_1 = 5$ и т.д.	не менее одного комментария с 4 потомками, не менее двух комментариев с 3 потомками, комментарии с одним потомком, комментарии без потомков (следует хранить не только id комментариев, но и сами комментарии).
4	p=03.08, q=Business	q = 7, p = «П»	Сумма 15 слагаемых $n / (n + 3)$ при начальном $n_0 = 4$, с шагом 1: $n_1 = 5$ и т.д.	как в варианте 1 - по структуре, не по содержанию!
5	p=10.08, q=Economy	q = 8, p = «Б»	Сумма двадцати слагаемых $n / (n + 2)$ при начальном $n_0 = 1$, с шагом 5: $n_1 = 6$ и т.д.	как в варианте 2 - по структуре, не по содержанию!
6	p=15.08, q=Comfort	q = 9, p = «М»	Сумма девяти слагаемых $n + n^4 + n^5$ при начальном $n_0 = 2$, с шагом 1: $n_1 = 3$ и т.д.	как в варианте 3 - по структуре, не по содержанию!
7	p=15.07, q=Business	q = 9, p = «В»	Сумма 13 слагаемых $n / (n + 2.5)$ при начальном $n_0 = 6$, с шагом 1: $n_1 = 7$ и т.д.	как в варианте 1 - по структуре, не по содержанию!
8	p=17.07, q=Economy	q = 8, p = «С»	Сумма семи величин $n \cdot (n + 3)$ при начальном $n_0 = 1$, с шагом 3: $n_1 = 4$ и т.д.	как в варианте 2 - по структуре, не по содержанию!
9	p=20.07, q=Comfort	q = 7, p = «К»	Сумма 14 слагаемых $n + n^2 + n^4$ при начальном $n_0 = 1$, с шагом 2: $n_1 = 3$ и т.д.	как в варианте 3 - по структуре, не по содержанию!
10	p=18.08, q = Business	q = 8, p = «А»	Сумма 11 слагаемых $n + n^{1.5} + n^{3.5}$ при начальном $n_0 = 2$, с шагом 2: $n_1 = 4$ и т.д.	как в варианте 2 - по структуре, не по содержанию!

Упражнения (блок 3 – представления, общие задания).

1. Создайте обычное представление на основе запроса, включающего внутреннее соединение хотя бы двух таблиц (придумайте запрос). В запросе (где-либо) должен присутствовать номер Вашего варианта.
2. Создайте пустое материализованное представление на основе любого запроса к БД. Выполните запрос для заполнения этого представления данными. В запросе (где-либо) должен присутствовать номер Вашего варианта.

